

PocketDRS

Defense Preparation Guide

Explained like you have never seen the code

Read me first. This guide does *not* use jargon.

Every idea is explained with an everyday example.

Every answer is short enough to say out loud in 20 seconds.

If something here uses a word you do not know, that is a bug in this document, tell us.

Niraj Kafle

University ID: LC0003001674

Supervisor: **Mr. Saishab Bhattarai**

May, 2026

BIT 7th Semester

Phoenix College of Management

Use this with the report and the slides. The report has the formal details. The slides are what the panel sees. This document is what you need to actually understand and speak about the project.

Contents

1	What This Whole Project Is, In One Page	2
2	The 60-Second Pitch (Memorise This)	3
3	The Whole Algorithm, Like You Are Five	3
3.1	Step 1, “Where was the phone?”	3
3.2	Step 2, “Where is the ball in each frame?”	3
3.3	Step 3, “Join the dots into one ball path”	4
3.4	Step 4, “Lift the path into 3D”	5
3.5	Step 5, “Apply the three LBW rules”	5
4	How the App Looks From the Outside (User Journey)	6
5	Defense Q&A, Whys, Hows, and Why-nots	6
5.1	A. Why does this project exist?	6
5.2	B. Why these technologies?	7
5.3	C. Step 1, Camera Pose, in detail	8
5.4	D. Step 2, Ball Detection, in detail	9
5.5	E. Step 3, Trajectory Association, in detail	10
5.6	F. Step 4, 3D Reconstruction, in detail	10
5.7	G. Step 5, LBW Decision, in detail	11
5.8	H. Results and Validation	12
5.9	I. Honest limits, what does NOT work	13
5.10	J. Engineering practice	14
5.11	K. Future work	14
6	Technical Glossary: Every Term Explained	15
6.1	Computer Vision Foundations	15
6.2	The Algorithms by Name	16
6.3	Detection Algorithms	16
6.4	Cricket Domain Terms	17
6.5	Backend / Mobile Stack	18

6.6	Software Quality Terms	19
7	Numbers and Facts Cheat Sheet	19
8	Trick Questions and How to Answer Them	21
9	Final Checklist Before You Walk In	22

1 What This Whole Project Is, In One Page

Cricket has a problem. When the ball hits the batter's leg before hitting the bat, the umpire has to guess where the ball *would have gone* next. Was it heading for the stumps? That guess is called **LBW** (Leg Before Wicket), and it is the hardest call in cricket.

The big leagues solved this with **Hawk-Eye**: six to eight expensive cameras around the ground, all wired together, all looking at the ball from different angles. From all those views they rebuild the ball's path in 3D and show where it would have gone. It works beautifully, but it costs more than a school's whole yearly budget.

So at every level *below* the international one, clubs, schools, training nets, nobody has this. The umpire just guesses.

PocketDRS is for those people. Instead of eight cameras and a control van, it uses *one* thing: a normal smartphone. You record the delivery, tap the four corners of the pitch on your phone screen (so the computer knows what real-world distances look like), and send the clip to our server. The server figures out where the camera was, finds the ball in each frame, fits a smooth curve through those positions, lifts that curve into 3D, then checks the three LBW rules and gives you a verdict: OUT, NOT OUT, or UMPIRE'S CALL. You see it as a 3D animation on your phone.

In everyday words

Hawk-Eye is like having eight friends watching a thrown ball from eight different sides, they can pinpoint exactly where it went. PocketDRS is like having *one* friend who watches really carefully and uses what they know (the ball is round and always the same size, the pitch is always 22 yards long, things fall because of gravity) to reconstruct what probably happened.

Does it work? On 66 fake-but-realistic deliveries (a computer-simulated cricketer) we got the right verdict 80% of the time, and *every single OUT was correct*. On a real phone clip filmed at a practice net it produced a clean ball track. On a badly-filmed real clip (camera too low, ball hidden behind the bowler's body) it refused to invent a verdict, which is exactly what you want a serious tool to do.

What it is not: It is not Hawk-Eye. It is not certified by the ICC. It is not a replacement for the umpire. It is a *decision-support tool*, like a calculator for an accountant. The umpire still calls the game.

2 The 60-Second Pitch (Memorise This)

“Hawk-Eye needs eight synchronised cameras and a control rig, way too expensive for clubs or schools. PocketDRS does the same job for LBW with just one phone camera. You tap the four pitch corners so the system knows real-world distances, then it finds the ball in each frame, joins those positions into one smooth path, lifts that path into 3D using the fact that the ball is always the same size and gravity always pulls down, and checks the three LBW rules. On 66 simulated deliveries it gets the right answer 80% of the time, including 100% of OUTs. On real handheld footage it tracks the ball cleanly when the camera is positioned well, and it safely refuses to guess when it cannot see the ball. It is decision support, not a replacement for the umpire.”

3 The Whole Algorithm, Like You Are Five

The server does five things, in order. That is the entire algorithm.

3.1 Step 1, “Where was the phone?”

When you tap the four corners of the pitch on your phone screen, the computer compares the *shape* of the pitch on the screen with the *real shape* of a cricket pitch (which is always 22 yards long and about 3 metres wide). From the difference between the two shapes, it figures out where the phone must have been standing and which direction it was pointing.

In everyday words

Imagine a friend draws a photo of a soccer goal on a napkin. If the goal looks like a flat rectangle, your friend was standing directly in front of it. If the right side looks bigger than the left, your friend was standing to the left. From the shape *in the picture* you can work out where the photographer was. The computer does exactly that with the cricket pitch.

Tricky bit: the maths has two possible answers, one where the camera was *above* the pitch (real) and a mirror version where the camera was *below* the pitch (impossible, because a camera underground is silly). We just throw away the silly answer.

3.2 Step 2, “Where is the ball in each frame?”

A video is just a stack of still pictures (frames). For each frame, we look for the ball in two different ways at the same time:

- **Things that moved.** Compare this frame with a model of what the background looks like when nothing is happening. Whatever has changed must be something moving, possibly the ball.
- **Things that are the right colour.** The ball is red (or white). Look for red-ish (or white-ish) patches in the frame.

If both methods agree on the same spot, “something red moved here”, we are pretty confident that is the ball.

In everyday words

Imagine looking for a red apple in a busy fruit market. One person looks for *anything that just moved*; another looks for *anything that is red*. When both say “it is over there” you trust them. When only one shouts, you are less sure.

One important trick. Some red things sit in the same place the whole video, a logo on a sign, a red helmet, a red bag. They are not the ball. The real ball only passes through any spot once or twice. So we remove any “red moving thing” that shows up in the *same place* in more than 30% of the frames. This one trick is what stops the system from falsely drawing a ball path through a cluster of background objects.

Another trick. A fast-moving ball does not look like a clean circle in a single frame, it looks like a short *streak* (like a comet tail). So we accept both shapes (circles and short streaks) as possible balls.

Real videos: for handheld phone clips we also use a smart detector called **YOLO** that has been trained on many cricket-ball photos and recognises cricket balls automatically.

3.3 Step 3, “Join the dots into one ball path”

After Step 2 we have a pile of “maybe-ball” positions in different frames. Some are the real ball, some are false alarms. We need the *one path* that makes sense.

We use a technique called **RANSAC**, which is just a fancy name for “try every guess and keep the best one”. Take two positions, draw the smooth curve a ball would follow between them, then count how many of the other positions land on or near that curve. Repeat thousands of times with different starting pairs. The curve with the most “it lands on me” votes wins.

In everyday words

Imagine someone scattered breadcrumbs and a few random pebbles across your lawn, and you want to find the path the bread-thrower walked. You pick two crumbs, draw a smooth line between them, and check: “how many *other* crumbs are near this line?” Try with different starting pairs. The line that explains the most crumbs is the path.

Two safety checks here:

1. If the “best path” barely moves across the picture (less than 20% of the image width), we throw it away, it is leftover clutter, not a real ball flying down the pitch.
2. If the curve does not fit smoothly enough (fit error bigger than 0.75 metres), we throw it away too.

In either case we report “no clear ball trajectory” instead of inventing a result.

3.4 Step 4, “Lift the path into 3D”

A photo is flat (2D). The world is 3D. How do we know how *far away* the ball was in each frame from one photo?

Two tricks:

1. **Depth from size.** A cricket ball is always exactly 36 millimetres in radius (the rules say so). If it looks big in the photo it is close; if it looks tiny it is far. Measure the ball’s size in pixels and you have a rough estimate of how far away it is.
2. **Physics.** A flying ball does not zig-zag, it follows a smooth curve shaped by gravity (it falls). So whatever rough 3D points we get from trick 1, we fit them to the curve gravity actually produces. This smooths out measurement noise.

One more trick: the bounce. When the ball touches the pitch, we know two things for sure: its height is exactly zero (it is on the ground) and its position on the pitch is directly under that pixel. We use that single rock-solid point to anchor the whole curve in place.

In everyday words

Imagine someone throws a tennis ball across a room and you only get a side-on photo. You can guess how far the ball is from you in each frame because you know how big a tennis ball is, if it looks tiny, it is far. Then you remember balls follow a curve because of gravity, so you smooth your estimates into a curve. And the moment the ball touches the floor is the one point you can locate *exactly*, so you pin your curve to that point.

3.5 Step 5, “Apply the three LBW rules”

The LBW law in cricket has three checks. The ball is OUT only if *all three* are true:

1. **Pitched in line.** Did the ball bounce in a sensible place (in line with the stumps, not way outside leg)?
2. **Impact in line.** Did the ball then hit the batter’s leg in line with the stumps (not outside off)?
3. **Hitting stumps.** Would the ball, if the leg had not been there, have gone on to hit the stumps?

If all three pass: **OUT**. If the ball would have just barely clipped the stumps (within 2.5 cm of the edge): **UMPIRE’S CALL**. Otherwise: **NOT OUT**.

In everyday words

Think of the bowler as someone throwing a ball at a small target (the stumps) past an obstacle (the batter’s leg). LBW asks: would the ball have hit the target if the obstacle had not been there? You answer that by extending the ball’s path past the leg and checking if it intersects the target.

4 How the App Looks From the Outside (User Journey)

1. **Sign in.** Email/password or Google. (Firebase does this for us.)
2. **New analysis.** Tap a button. Pick a video from your phone or record one.
3. **Trim the clip.** Drag two handles so the clip starts just before the delivery and ends just after the bounce. About 3–5 seconds.
4. **Calibrate the pitch.** On the first frame, tap the four corners of the pitch (left-near, right-near, right-far, left-far). The app draws lines to show the corners are joined into a quadrilateral.
5. **Submit.** The clip + the tap positions go to the server. You see a progress bar.
6. **Result.** A 3D viewer opens. You can spin the view around. Below it: the verdict (OUT / NOT OUT / UMPIRE’S CALL) with a one-sentence reason.
7. **History.** Tap “Home”. All past analyses are listed.

That is the entire user experience. Five taps, one wait, one verdict.

5 Defense Q&A, Whys, Hows, and Why-nots

This is the heart of the document. Read each question, read the answer, then say it out loud in your own words once.

5.1 A. Why does this project exist?

Q. Why did you build PocketDRS?

Because cricket review technology only exists at the top level. Hawk-Eye costs more than most clubs earn in a year. Below the international level there is no review aid at all, and the same close LBW calls happen every weekend. We wanted to see if a single normal phone could give those people *some* kind of help.

Q. Why LBW and not run-outs or stumpings?

LBW is the only decision where you have to imagine where the ball *would have gone next*. A run-out or a stumping is about where things *actually are*. LBW needs a 3D ball path. That makes it the hardest *and* the most useful thing to build a tool for.

Q. Is this just a Hawk-Eye copy?

No. Hawk-Eye uses eight cameras and triangulates depth like your two eyes do. We use one camera and have to be clever about depth (ball size + physics). Hawk-Eye is accurate to a few millimetres on a controlled field. We are accurate to about half a metre on synthetic tests and refuse to guess when the footage is bad. Different problem, different solution, different audience.

Q. Why not just license a Hawk-Eye SDK?

Hawk-Eye is closed and only sold to broadcasters. There is no SDK to license. Even if there were, it would need eight cameras installed, defeating the whole point.

5.2 B. Why these technologies?**Q. Why a phone app plus a cloud server, not just on the phone?**

Three reasons:

1. Phones vary wildly in CPU strength. Doing the heavy work on a server we control is more reliable.
2. Putting analysis history in the cloud lets users switch phones without losing their data.
3. We can improve the algorithm whenever we like, without forcing every user to update the app.

On-device analysis is on our future-work list once everything is stable.

Q. Why Flutter for the app?

One codebase makes both Android and iOS apps. One developer can build both. It has good plugins for the camera, video, and Firebase. The alternative (writing two native apps) would have doubled the work for no real benefit.

Q. Why FastAPI for the server?

It is the lightest Python web framework that is fast, modern, and works well with the computer-vision libraries we need (OpenCV, NumPy, SciPy). Django would be overkill (it expects a database). Flask is older and slower.

Q. Why Firebase / Firestore for the database?

The data is per-user and document-shaped, not table-shaped. Firestore gives us logins, security rules, and live progress updates for free. Building those with a SQL database would have taken weeks.

Q. Why Three.js for the 3D viewer?

It runs inside a WebView on both Android and iOS with no extra plugins, and looks identical on both. Native 3D plugins for Flutter are immature. Plus we could ship the same viewer as a web page if we wanted.

Q. Why Google Cloud Run, not a normal server?

Cloud Run scales down to zero when nobody is using the system, so we pay nothing when idle. It scales up automatically when many people upload at once. Perfect for a student project that may have bursts of demo traffic.

5.3 C. Step 1, Camera Pose, in detail**Q. Why does the user have to tap four corners, can the system not find them by itself?**

It could in principle (using line detection), but doing it reliably across every ground, every lighting condition, and every camera angle is its own research project. Asking the user to tap four corners takes 30 seconds and is much more reliable. Automatic detection is on our future-work list.

Q. What is PnP in plain words?

PnP just stands for “Perspective-n-Point”. Given a few real-world points and where they appear on the photo, it figures out where the photographer was standing. We give it the four pitch corners (real positions) and the four taps (where they show up on the photo), and it gives back the camera’s position and direction.

Q. What is the “twofold mirror” problem?

The maths has two equally-good answers: one where the camera is above the pitch (correct) and a mirror version where the camera is below the ground (silly). We just check which one is above ground and keep that.

Q. Why do you assume a 67° field of view?

That is roughly the angle of view of a normal smartphone main camera. We could measure it per phone, but that would mean asking the user to film a checkerboard pattern first, which would be annoying. The small error from assuming 67° is much less than the depth-from-size error, which dominates the final accuracy anyway.

5.4 D. Step 2, Ball Detection, in detail**Q. Why use motion AND colour, not just one?**

Each on its own gives lots of false alarms. Motion alone fires on *anything* that moves: players, swaying nets, shadows. Colour alone fires on *anything* that is the right shade: a red helmet, a red logo. Requiring both to agree cuts the false alarms while still finding the real ball.

Q. What is MOG2 (motion detection)?

MOG2 is the standard OpenCV background detector. It learns what the background looks like over time (allowing for slow lighting changes), then flags anything that is different from that background as “moving”. It handles swaying nets and mild camera shake better than simpler methods.

Q. Why look for “streaks” as well as circles?

A fast ball at 30 frames per second *smears* across a frame. It looks like a short streak, not a clean circle. If we only looked for circles we would miss every fast delivery.

Q. What is fixed-clutter suppression and why is it your most important fix?

Anything that shows up in the same pixel location in more than 30% of frames is removed. Real balls pass through any location once or twice. Logos, helmets, and bags sit still and would fire every frame. Without this fix the next step would happily “find” a ball path through that static clutter. With this fix it cannot.

Q. What is YOLO and when do you use it?

YOLO is a deep-learning detector that has been trained on lots of cricket-ball photos. It is more reliable on messy real-world video than our motion+colour combination, but slower. We use it on real handheld clips; the motion+colour combination is enough on cleaner footage.

5.5 E. Step 3, Trajectory Association, in detail

Q. Why RANSAC and not a Kalman filter?

RANSAC looks at the whole clip at once and finds the single best path. A Kalman filter goes frame by frame and assumes you already know where to start. For our “analyse this 3-second clip” workflow RANSAC is simpler, more robust to background junk, and easier to debug. We mention Kalman in the report because it is the natural next step for live tracking.

Q. Why measure total movement across the clip, not frame-to-frame speed?

Because the camera is behind the bowler, the ball is flying *towards* the camera. Frame-to-frame it does not seem to move much across the screen. Measuring how much it moved from start to end of the clip is a much better way to tell a real delivery from a stationary false alarm.

Q. Why throw away tracks that cover less than 20% of the image?

Because a real delivery sweeps across most of the picture. A short tight cluster of points is almost always background clutter. This is one of the safety checks that makes the system say “no trajectory” instead of inventing one.

5.6 F. Step 4, 3D Reconstruction, in detail

Q. How can one camera see depth at all?

It cannot directly. We add two pieces of outside knowledge: (1) the ball is always the same size (36 mm radius), so if it looks tiny in the picture it must be far away; (2) the ball is flying, so its path must be a smooth curve shaped by gravity. Together these two constraints give us enough information to recover a reasonable 3D path from one camera.

Q. Why anchor the curve at the bounce?

Because the bounce is the one moment when we know *exactly* where the ball is in 3D: its height is zero (touching the pitch), and we can project the bounce pixel onto the pitch plane to get its forward/sideways position. Anchoring the curve to this rock-solid point removes a huge amount of scale uncertainty.

Q. Why cap the release height and forward speed in the fit?

Without limits, the maths can wander off into impossible solutions (a ball launched from 12 metres high travelling sideways at 200 km/h). Limiting the search to physically realistic values forces the fit to stay close to a real-world delivery.

Q. Why ignore air resistance and spin?

Over the ~ 20 m flight of a delivery, air resistance and spin push the ball by less than 1 cm. Our depth measurement is already noisy by tens of centimetres. So modelling drag and spin would add complexity without improving the answer. Listed in future work for high-precision use cases.

Q. What is the 0.75-metre quality gate?

A safety latch. If the curve we fit is more than 0.75 metres off the measured points on average, something is wrong, we probably did not actually find the ball. So we throw the whole reconstruction away and tell the user “no clear trajectory”. This stops the system producing confident-but-wrong verdicts on bad input.

5.7 G. Step 5, LBW Decision, in detail**Q. What does ICC Rule 36 say, in three sentences?**

(1) The ball must *pitch* in line with the stumps (not way outside leg). (2) The ball must *strike the batter* in line with the stumps. (3) The ball’s projected path must *go on to hit the stumps*. All three true $\rightarrow$ OUT.

Q. Why is the umpire’s-call margin 25 mm?

That is the international standard. Hawk-Eye uses it. The idea is: if the projected ball is so close to the stump edge that even a small measurement error could flip the verdict, leave the call to the human umpire. We use the same number so our verdicts read the way an umpire expects.

Q. Why widen the vertical margin to 7.2 cm?

Our depth measurement is noisier in the up-down direction at the stumps than left-right. Using the same tight 2.5 cm margin vertically would unfairly push real OUTs into umpire’s-call. One ball-diameter (7.2 cm) is a fair compromise that matches our actual precision.

5.8 H. Results and Validation

Q. Why use a synthetic test as well as real video?

Because synthetic tests have *ground truth*, we built the simulator, so we know exactly where the ball went. That lets us measure accuracy with numbers. Real video does not have ground truth, we cannot know the correct answer with certainty, so we cannot compute a percentage on it. The two tests answer different questions: synthetic says “how often is the algorithm right?”; real says “does it work on a phone in the wild?”.

Q. Walk me through the 53 / 66 result.

We simulated 3 ball speeds (18, 24, 30 m/s) $\times$ 11 release directions (clearly wide of off, off-stump line, middle, leg-stump, outside leg, going down) $\times$ 2 release angles = 66 deliveries. The system gave the right verdict on 53 of them (80.3%). Per category: OUT 16/16 (100%), NOT OUT 36/44 (81.8%), UMPIRE’S CALL 1/6. The 13 failures all came from very fast deliveries (few frames) or very close calls (within ± 20 cm of the line), both inherent limits of a single camera.

Q. Why is the OUT class your strongest result?

Because OUT needs *all three* checks to pass. That demands a coherent end-to-end trajectory, which our physics-anchored fit produces reliably. OUTs are also the most consequential verdict for an umpire to get right, being perfect there is the most important guarantee.

Q. What happened with the first real video?

Four things went wrong at once: (1) the file header said it was 7 seconds long, but only 3.25 seconds actually decoded; (2) the camera was at chest height, so the bowler’s body blocked 20–36% of the view during the delivery; (3) it was filmed through green net mesh; (4) the background had swaying nets, trees, and two moving people. The system correctly returned “no clear ball trajectory”. The first item was a real software bug (we fixed it). The others are setup mistakes that the app now warns about.

Q. Why is a “no verdict” a good result?

Because a tool that confidently invents wrong answers is worse than useless, it actively misleads umpires. A tool that honestly says “I cannot see the ball in this clip” is something you can trust. The bad-clip behaviour is exactly what a safe decision-support tool should do.

Q. What about the second real video (test3)?

1080p / 60 fps handheld clip of an off-spin (flighted) delivery at indoor nets. Because off-spin loops up high, the ball stayed visible above the bowler's arm. The YOLO detector found ball candidates on every readable frame; the new truncation guard stopped at the last real frame (101) even though the header claimed 290; RANSAC linked 22 inliers out of 33 candidates with a 14.4-pixel fit error; a recurring red false-positive was correctly removed by fixed-clutter suppression. The 3D lift produced a smooth descending arc with the correct shape. Absolute distance is monocular-ambiguous and we report it that way, we do not pretend to know the speed precisely.

5.9 I. Honest limits, what does NOT work**Q. So is this Hawk-Eye?**

No. Hawk-Eye uses 6–8 synchronised cameras and gets 3–5 mm accuracy under controlled conditions. We use one camera and get about half a metre. Hawk-Eye is for international cricket; PocketDRS is for clubs, schools, and nets. Same goal, very different precision.

Q. Why is your accuracy half a metre instead of millimetres?

Because one camera cannot triangulate the way two eyes can. We estimate depth from how big the ball looks, which is noisy: at 10 m away the ball is only 3 pixels across, so a 1-pixel error means a 10% depth error. The only way to fix this is more cameras, exactly the missing ingredient by design.

Q. When does the system fail?

Three situations: (1) visibility (ball hidden by the bowler, net mesh, or out of focus); (2) calibration (taps far from the real pitch corners); (3) speed (super-fast deliveries that only appear in a handful of frames). In all three cases the system *fails safely*, it returns a warning, not a fake verdict.

Q. What if the user taps the corners wrong?

Two safety nets. (1) The app checks that the four taps form a sensible four-sided shape (not all the same point, not all on a straight line). (2) The server checks the reprojection error of the recovered camera, if it is too high, it warns the user.

5.10 J. Engineering practice

Q. How is the code organised?

`server/app/` is the Python backend (FastAPI + one module per algorithm step: video reader, detector, tracker, calibration, reconstruction, job-runner). `app/pocket_drs/` is the Flutter mobile app. Each algorithm step is a small pure function so we can unit-test it on its own.

Q. How many tests do you have?

20 unit tests (per module) and 12 system tests (end-to-end). All pass. Unit tests cover every individual function; system tests cover every realistic scenario including all three verdict types, the bad real clip, the good real clip, and the safety-net behaviours.

Q. How do you deploy?

Docker builds a container, `gcloud run deploy` pushes it to Cloud Run, Firestore and Cloud Storage are already provisioned. The mobile app is built with `flutter build`.

Q. What about data privacy?

All data lives under the user's Firebase user-ID. Security rules stop other users from reading it. Data is encrypted in transit (HTTPS). We do not share with third parties and we honour deletion requests. This is documented in NFR-08.

5.11 K. Future work

Q. What is the single most impactful next improvement?

Two phones. Even two phones triangulating the ball would remove the depth-from-size problem entirely and bring our accuracy much closer to multi-camera DRS, at zero extra hardware cost beyond a second phone.

Q. Could the whole pipeline run on the phone someday?

Yes. The detector and trajectory step are already feasible on modern phones (YOLO runs at 30 fps on flagship hardware). The 3D fit and LBW logic could follow. We deliberately designed the boundary as “frames in, JSON out” so we can move that boundary on-device later without rewriting the mobile app.

Q. Could you do live (in-match) analysis?

Possible but harder. The current system is post-hoc, you record a clip, then analyse it. Live analysis means a streaming detector, per-frame state tracking (this is where Kalman comes back), and continuous verdict refresh. Natural next research project on top of this one.

6 Technical Glossary: Every Term Explained

If the panel asks “what does X mean?” your answer lives here. One paragraph per term, in plain English, with the precise definition at the end.

6.1 Computer Vision Foundations

Pixel. The smallest dot of colour in a digital image. A 1080×1920 frame has roughly 2 million pixels. Everything the computer “sees” in a video is just a grid of pixel colour values.

Frame. One still picture inside a video. At 30 frames per second (fps) the camera captures 30 stills every second; at 60 fps it captures 60.

Pinhole camera model. The mathematical approximation that says: light from every 3D point in the world passes through a single point (the lens) and lands on a flat sensor. It is the standard model used everywhere in computer vision because it is simple, accurate enough, and fully described by a small set of numbers.

Intrinsic camera parameters. The numbers that describe how a particular camera maps 3D points onto its sensor. Focal length (f_x, f_y) controls how “zoomed in” the camera is, and principal point (c_x, c_y) is where the centre of the lens lines up on the sensor. Together they make the camera matrix K .

Extrinsic camera parameters. Where the camera is and which way it points. Two pieces: a rotation R (orientation) and a translation t (position). Together they place the camera in the world.

Field of view (FOV). The angle the camera can see, measured in degrees. A typical phone main camera has roughly 67° horizontal FOV.

Reprojection error. A quality check. Take a known 3D point, project it through your camera matrix, and compare the predicted pixel with the actual pixel. The distance between them, measured in pixels, is the reprojection error. Smaller is better; PocketDRS gets 0.00 px on synthetic data and a few pixels on real clips.

Homography. A 3×3 matrix H that maps points between two flat surfaces seen from different angles. Because a cricket pitch is flat, one homography converts any pixel inside the pitch into a real-world position on the pitch in metres.

Camera calibration. The process of measuring the intrinsic parameters of a specific camera. The classical method (Zhang, 2000) uses photos of a checkerboard pattern. PocketDRS skips this step and assumes typical smartphone values so users do not have to film a checkerboard.

6.2 The Algorithms by Name

PnP (Perspective-n-Point). The algorithm that recovers the camera’s position and orientation when you know a few real-world 3D points and where they appear on the photo. PocketDRS feeds it the four pitch corners (real positions, in metres) plus where the user tapped them (pixels), and gets back the camera’s pose. We use OpenCV’s `solvePnP` with the ITERATIVE solver because it accepts the planar case.

Twofold mirror ambiguity (planar PnP). A quirk of the maths: when all the known points lie on one flat surface, PnP has two equally good solutions, one with the camera above the surface and one with it below (mirrored). The maths cannot tell them apart. We just reject the answer where the camera is underground.

RANSAC (Random Sample Consensus). A robust way to fit a model when your data has outliers. Pick the smallest random sample needed to define the model, fit the model to it, count how many of the remaining points agree (“inliers”), and keep the model with the most inliers. PocketDRS uses it twice: once to compute the homography from the four pitch taps, once to link per-frame ball detections into a single trajectory.

Inliers vs outliers. Inliers are measurements that agree with the model you fitted. Outliers are measurements that do not. RANSAC’s job is to find the largest possible inlier set.

Projectile motion. The physics of a thrown object under gravity alone (ignoring air resistance). Horizontal velocity stays constant; vertical velocity decreases by 9.81 m/s every second. The trajectory is a parabola. Cricket-ball flight is close enough to projectile motion that we can use it as a strong constraint.

Bundle adjustment. A classical computer-vision technique that jointly refines camera parameters and 3D point positions to minimise reprojection error across many views. PocketDRS borrows the same least-squares philosophy for the projectile-fit step but with only one view.

Coefficient of restitution. A number between 0 and 1 that describes how bouncy a collision is. After bouncing on the pitch, a cricket ball keeps about 55% of its vertical speed; the coefficient of restitution is therefore ≈ 0.55 . We use this in the bounce model.

Depth-from-size. The trick we use to recover distance from a single camera. Because a cricket ball is always exactly 36 mm radius, its size in pixels tells you how far away it is. A small ball in the image must be far away.

Kalman filter / Extended Kalman Filter (EKF). A standard recursive estimator for tracking a moving object frame by frame. It combines a motion model (where do we expect the object to be next?) with measurements (where did we observe it?) and keeps an uncertainty estimate. PocketDRS does not use Kalman in the final pipeline because RANSAC plus projectile-fit is simpler and more robust to clutter; Kalman is the natural next step for live tracking.

6.3 Detection Algorithms

MOG2 (Mixture of Gaussians 2). The standard OpenCV background-subtraction algorithm. It models each pixel of the background as a mixture of several Gaussians (so it can handle slow lighting drift and mild swaying) and flags anything that does not fit the background model as “moving”. We use it for

motion detection.

HSV colour space. An alternative way of describing colour (Hue, Saturation, Value) that separates “what colour” from “how bright”. Detecting the red of a cricket ball is much easier in HSV than in RGB because shadows mostly affect Value while leaving Hue alone.

ROI (Region of Interest). A rectangular area of the image where you focus your search. We use ROIs to ignore obvious non-pitch areas (e.g. the sky, the umpire) before running the detector.

YOLO (You Only Look Once). A deep-learning object detector that scans the whole image in a single pass and outputs bounding boxes around the objects it recognises. The name comes from how it only looks at the image once (older detectors scanned thousands of windows). It is fast and accurate, so it has become the default real-time object detector in computer vision.

Ultralytics YOLOv8. The specific implementation of YOLO we use. Ultralytics is the company that maintains the modern PyTorch-based version. We feed it a fine-tuned cricket-ball model so it only detects cricket balls (and ignores everything else).

Motion blur / streak detector. A fast object in a single frame does not look like a clean shape, it smears into a short streak (like a comet tail). Our motion detector explicitly accepts both round blobs and short streaks as possible ball candidates, so it does not miss fast deliveries.

Fixed-clutter suppression. Our custom filter that removes any detection appearing in the same pixel location in more than 30% of frames. The reasoning: a real ball passes through any spot at most twice; a logo, helmet, or bag sits in one place for the whole clip.

6.4 Cricket Domain Terms

LBW (Leg Before Wicket). A way for a batter to be dismissed. The ball hits the batter’s leg in a position where it would have gone on to hit the stumps if the leg had not been in the way. The umpire’s hardest decision.

ICC (International Cricket Council). The world governing body for cricket. They publish the Laws and the Playing Conditions, and they certify ball-tracking systems for international use (Hawk-Eye is ICC-certified; PocketDRS is not).

ICC Rule 36 / Law 36. The specific cricket law that defines LBW. It has three checks: pitched in line, impact in line, wicket hitting. All three must be true for an OUT.

Pitched in line. The ball must bounce in line with the stumps (or outside off stump). If it bounces outside the leg-stump line, it cannot be LBW.

Impact in line. The ball must hit the batter in line with the stumps. If the impact is outside off, the batter cannot be out LBW (unless they offered no shot).

Wicket hitting (hitting stumps). The projected continuation of the ball’s path after the impact must intersect the stumps.

Umpire’s call. The buffer zone used by Hawk-Eye and the ICC: if any of the three checks is closer than about half a ball-radius to the boundary, the on-field umpire’s original decision stands. We use 25 mm horizontally and 7.2 cm vertically.

Stumps. The three vertical wooden poles at each end of the pitch. Combined width of the three is roughly 22.86 cm; each stump is about 71.1 cm tall.

Pitch. The 22-yard strip (20.12 m) between the two sets of stumps. The bowler bowls from one end, the batter stands at the other.

Crease. The line painted across the pitch in front of each set of stumps; the batter stands behind it.

Off side / Leg side. The two halves of the pitch from the batter's point of view. For a right-handed batter, off side is on the right; leg side is on the left. "Outside off" and "outside leg" are how umpires describe where the ball pitched or struck.

Hawk-Eye. The professional broadcast ball-tracking system used in international cricket since the early 2000s. Uses 6 to 8 synchronised high-speed cameras, ICC-certified, accurate to a few millimetres on controlled fields.

Flighted delivery. A delivery (usually slow spin) that the bowler throws up in a loop so it descends from above onto the batter. Important for PocketDRS because the higher loop means the ball stays visible above the bowler's body in our real clip.

6.5 Backend / Mobile Stack

Flutter. Google's cross-platform mobile framework. Write one Dart codebase, get both Android and iOS apps. We use Flutter for the mobile client.

Dart. The programming language used by Flutter. Object-oriented, compiles to native code or JavaScript.

FastAPI. A modern, fast Python web framework with built-in async support and automatic OpenAPI documentation. We use it for the backend.

Async / await. A way of writing code that waits for slow operations (like network or disk) without blocking other work. Important because video processing is slow; FastAPI's async support keeps the API responsive while the pipeline runs in the background.

REST API. A style of web service where the client makes HTTP requests (GET, POST, etc.) and the server returns JSON. Our mobile app talks to the FastAPI server using REST.

JSON. A simple text format for structured data (lists, dictionaries, numbers, strings). Used for every request and response between the app and the server.

OpenCV. The standard open-source computer-vision library. We use it for every image and video operation: decoding frames, motion detection, drawing the result overlay, computing homographies, running PnP.

NumPy. The Python library for fast numerical arrays. Almost every vision operation works on NumPy arrays under the hood.

SciPy. The Python library for scientific computing. We use `scipy.optimize` for the least-squares projectile fit.

Three.js. A JavaScript library for 3D graphics in the browser. We use it to render the interactive Hawk-Eye-style 3D viewer inside a Flutter WebView.

WebView. A web-browser view embedded inside a mobile app. Lets us run our Three.js viewer on both Android and iOS without writing native 3D code twice.

Firebase Authentication. Google's hosted user-login service. Handles email/password, Google sign-in, token refresh, and password reset for us.

Firestore. Google's hosted NoSQL document database. We store per-user analyses and pitch calibrations here. Comes with real-time sync, security rules, and live updates.

Cloud Storage. Google's hosted file storage. We keep the trimmed video files here while a job is being processed.

Cloud Run. Google's serverless container hosting platform. Our backend container scales down to zero when nobody is using it (no idle cost) and scales up automatically under load.

Docker / container. A standard way to package an application with all its dependencies into one runnable image. Lets us deploy the backend identically on any machine.

6.6 Software Quality Terms

Unit test. A test that exercises one small piece of code in isolation. We have 20.

System test (end-to-end test). A test that exercises the full pipeline from input to output, like a real user would. We have 12.

Synthetic data / synthetic harness. Computer-generated test data with known ground truth. Lets us measure accuracy with numbers (impossible on real video where the truth is unknown).

Ground truth. The known correct answer. In synthetic data we built it ourselves so we know it exactly.

Quality gate. A check that automatically rejects a result when it looks wrong. PocketDRS rejects any 3D reconstruction whose fit error exceeds 0.75 m, so the system returns a warning instead of a confident-but-wrong verdict.

Edge case. A scenario at the boundary of valid input (truncated video, all-zero corners, NaN detections). Hardening is the act of finding and fixing how the system behaves on edge cases.

Fail-safe behaviour. Failing in a way that does not harm the user. For a decision-support tool, "no verdict" is fail-safe; "confident wrong verdict" is not.

7 Numbers and Facts Cheat Sheet

Keep this page open during the defense.

Physical constants

- Cricket ball radius: **0.036 m** (36 mm).
- Gravity: $g = 9.81 \text{ m/s}^2$.
- Coefficient of restitution (bounce energy retained): ≈ 0.55 .
- Pitch length (stumps to stumps): **20.12 m** (22 yards); width $\approx 3 \text{ m}$.
- Stump height: 71.1 cm; full stump width: $\approx 22.86 \text{ cm}$.
- Umpire's-call margin (horizontal): **25 mm**; vertical band used: **7.2 cm** (one ball diameter).

Camera & Calibration

- Assumed horizontal field of view: 67° (typical phone main camera).
- Synthetic camera position: $(-3.5, 0, 1.8) \text{ m}$ looking at $(10, 0, 0) \text{ m}$.
- Synthetic intrinsics: $f_x = f_y = 900$, $c_x = 540$, $c_y = 960$; 1080×1920 @ 60 fps.
- PnP reprojection error on synthetic: **0.00 px** (every scenario).

Detection & Tracking

- Candidates per synthetic scenario: 60–130.
- RANSAC inliers per scenario: 60–70 ($\sim 95\%$ of in-pitch detections).
- Image-space fit error (synthetic): 10–25 px.
- Fixed-clutter suppression threshold: **30%** of frames at the same location.
- Path-length sanity check: reject if track moves less than 20% of image width.
- Reconstruction quality gate: $\text{MAX_FIT_RMS_M} = 0.75 \text{ m}$.

Synthetic Results (66 deliveries)

- Scenarios: **66** ($3 \text{ speeds} \times 11 \text{ lines} \times 2 \text{ angles}$).
- Correct verdicts: **53 / 66 = 80.3%**.
- Per class: OUT **16/16 (100%)**, NOT OUT 36/44 (81.8%), UMPIRE'S CALL 1/6.

Real Clip (test3, success case)

- 1080×1920 @ 60 fps handheld; off-spin delivery; indoor nets.
- Container claims 290 frames; only 101 decode (truncation guard stops at 101).
- Tracked: **25** positions; **22 inliers / 33 candidates**; image-RMS **14.4 px**.
- Flight: frame 33 ($t = 550 \text{ ms}$) $\rightarrow$ frame 64 ($t = 1067 \text{ ms}$) = 517 ms tracked at 60 fps.

8 Trick Questions and How to Answer Them

Q. “This is just a Hawk-Eye clone.”

Hawk-Eye uses 6–8 synchronised high-speed cameras and triangulates depth directly. We use one consumer phone camera and recover depth from ball size + projectile physics. Hawk-Eye is accurate to a few millimetres on a controlled field. We are accurate to about half a metre on synthetic tests and we refuse to invent answers on bad real footage. Same goal, totally different solution, totally different audience.

Q. “80% accuracy is not very good.”

80% is the overall figure across a deliberately hard sweep that includes super-fast deliveries with very few frames and razor-close calls. On the most consequential class (OUT) we are at 100%. The 13 failures all sit on the well-understood single-camera depth-from-size limit, exactly the limit we wanted to document.

Q. “Your real video did not even work.”

The first real clip did not produce a track, and that is the *correct* behaviour on that clip (low camera, occlusion, netting). The system safely refused to invent a verdict. The second real clip, recorded with the guidance the app now provides, worked end-to-end: clean 25-point track, plausible 3D arc. The two clips together prove the system fails safely and succeeds when conditions are met.

Q. “Why didn’t you use machine learning for everything?”

Two reasons. (1) Physical constraints (pitch geometry + projectile motion) are stronger priors than any model could learn from the limited cricket data available. (2) ML-only systems hide their failure modes; a physics-based pipeline can be reasoned about, tested per stage, and made to fail safely. We do use ML where it actually pays off (YOLO detector for real clips). We do not use ML where the geometry already gives the right answer.

Q. “Could the user just trust the umpire?”

At the top level the umpire already has DRS. At club / school / coaching level the umpire has nothing, and the same close decisions happen. PocketDRS exists for that gap. It is decision support, not authority, the umpire still calls the game.

9 Final Checklist Before You Walk In

- Slides exported as PDF as a backup (in case PowerPoint mis-renders something).
- Phone with the app installed, signed in, with at least one saved analysis ready.
- Pre-loaded synthetic OUT, NOT OUT, and UMPIRE'S CALL renders to show if the live demo fails.
- Pre-loaded `test3.mp4` (success clip) and `test1` (failure clip) for the real-world story.
- A printed copy of: this PDF, the final report, and the IEEE reference list.
- Bottle of water on the desk; relax your shoulders; the panel wants you to succeed.

You know this work. You built it. Go tell the story.

Good luck, Niraj.